

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 - Redes

Sección 10

Kevin Velasquez



Lab 3 - Algoritmos de Enrutamiento

Diego Patzan - 23525

GUATEMALA, 7 de septiembre del 2026

1. Repositorio de GitHub

<https://github.com/UVG-Tareas-y-proyectos/RD-LAB03>

2. Descripción de los algoritmos y su implementación

Arquitectura común

Cuatro piezas sostienen los cuatro algoritmos. `packet.py` arma y lee el paquete estándar del protocolo (`proto`, `type`, `from`, `to`, `ttl`, `headers`, `payload`). Cada reenvío agrega un header `hop` con el id del nodo que reenvía, porque el campo `from` guarda el origen real del mensaje y no cambia salto a salto: sin un dato de "quién me lo mandó directamente" no se puede excluir al vecino correcto en flooding ni identificar de qué vecino vino un vector o un LSP.

`transport.py` es un servidor TCP que recibe líneas de texto separadas por salto de línea, cada una un paquete JSON. Enviar un paquete abre una conexión nueva, manda la línea y la cierra.

`routing/base.py` define la interfaz común (`on_hello`, `on_info`, `build_info_payloads`, `next_hop`, `get_forward_targets`) que `node.py` usa para tratar a los cuatro algoritmos de la misma forma, sin preguntar cuál es cuál.

`node.py` arma el nodo. El hilo de `routing` corre un loop que cada 5 segundos manda `hello` e `info` a los vecinos, mientras saca de una cola los paquetes `info` que van llegando; un lock protege la tabla para que no se mezclen escrituras desde distintos hilos.

Flooding

Es el algoritmo más simple: no arma tabla de ruteo. Cuando le llega un paquete de datos que no es para él, lo reenvía a todos los vecinos menos por el que le llegó (`broadcast_targets`). Para no reenviar el mismo mensaje indefinidamente en una red con ciclos, cada mensaje de usuario lleva un id único (UUID) en los headers; cada nodo recuerda los ids ya procesados (`seen_messages`, en `node.py`) y descarta los repetidos. El `ttl` actúa como segunda protección: se decrementa en cada salto y el paquete se descarta al llegar a cero.

Dijkstra

`shortest_paths(graph, source)`, en `dijkstra.py`, es la implementación en sí: una cola de prioridad (heapq) acumula distancias y, además de la distancia mínima, guarda el primer salto desde el origen hacia cada destino el dato que realmente necesita la tabla de ruteo, no el camino completo.

Esta función se usa de dos formas. Como algoritmo standalone (`DijkstraStatic`), lee la topología completa directamente del archivo `topo-*.txt` y calcula la tabla una sola vez al arrancar; el enunciado permite esta excepción porque Dijkstra puro es estático por definición y no tendría sentido descubrir vecinos dinámicamente para después ignorar esa información. Dentro de LSR, en cambio, se recalcula cada vez que cambia la base de datos de enlaces.

Link State Routing (LSR)

Cada nodo arma su propio LSP (Link State Packet: sus vecinos y el costo hacia ellos) con un número de secuencia creciente, y lo manda a todos sus vecinos. Al recibir un LSP, si el número de secuencia no es más nuevo que el guardado para ese origen, el nodo lo descarta así evita reenviar información vieja o entrar en bucle. Si es más nuevo, actualiza su base de datos de enlaces (`lsdb`) y lo reenvía a todos los vecinos menos por el que le llegó, reusando `broadcast_targets` de `flooding.py`, que es justo la parte que el enunciado pide reutilizar. Después

de actualizar la lsdb, recalcula la tabla completa corriendo `shortest_paths` sobre el grafo que arma esa base de datos.

Cada nodo termina así con una copia de la topología completa, ensamblada a partir de piezas que le llegaron de a poco, y calcula sus rutas con Dijkstra sobre esa copia sin haber leído nunca el archivo de topología.

Distance Vector Routing (DVR)

Cada nodo mantiene su propio vector de distancias (dist: destino -> costo), inicializado con el costo a sus vecinos directos, y manda ese vector completo a cada vecino en cada ronda. Al recibir el vector de un vecino aplica Bellman-Ford: costo nuevo es igual al costo hacia ese vecino más el costo que el vecino reporta hacia ese destino. Si el costo nuevo es menor que el que ya tenía, actualiza la distancia y marca a ese vecino como el siguiente salto para ese destino.

A diferencia de LSR, no hay reenvío de la información tal cual: cada nodo manda solo su propia vista actualizada en la siguiente ronda, y la información se propaga de a un salto por ronda.

Limitación conocida: no está implementado split horizon ni poison reverse, así que en una red más grande o con caídas de enlaces esto podría sufrir el problema clásico de count-to-infinity. Con la topología de prueba 5 nodos, sin caídas no se manifestó.

Costo de los enlaces

En las cuatro implementaciones el costo de un enlace está fijo en 1; no se mide el RTT real del hello para usarlo como costo. Queda anotado en el código como simplificación deliberada.

3. Topología usada en las pruebas

Las pruebas corrieron sobre `config/topo-example.txt`: un anillo de cuatro nodos (A, B, C, D) con un quinto nodo, E, colgado de A.

Nodo	Vecinos
A	B, D, E
B	A, C
C	B, D
D	A, C
E	A

Esta topología prueba tres cosas a la vez: caminos con más de un salto (A a C), ciclos para ver si el algoritmo entra en loop y un nodo con un solo vecino (E), que obliga a que todo lo que entra o sale de E pase por A.

4. Resultados

Pruebas unitarias de los algoritmos

`python tests/test_routing.py` corrió seis pruebas: cálculo de Dijkstra, convergencia de LSR con LSPs viejos y nuevos, convergencia de DVR, exclusión del vecino emisor en flooding, y manejo de headers del paquete. Las seis pasaron.

Prueba	Módulo cubierto	Resultado
test_dijkstra_shortest_path	dijkstra.py	ok
test_flooding_excludes_sender	flooding.py	ok
test_lsr_converges_to_full_graph	lsr.py	ok
test_lsr_ignores_stale_lsp	lsr.py	ok
test_dvr_updates_via_neighbor	dvr.py	ok
test_packet_headers	packet.py	ok

```
ok: test_dijkstra_shortest_path
ok: test_flooding_excludes_sender
ok: test_lsr_converges_to_full_graph
ok: test_lsr_ignores_stale_lsp
ok: test_dvr_updates_via_neighbor
ok: test_packet_headers
```

6 pruebas pasaron

presiona una tecla para cerrar

Flooding: mensaje sin tabla de ruteo

Con los cinco nodos levantados, se mandó un mensaje de E a C no vecinos directos, a tres saltos por el camino más corto. C imprimió el mensaje una sola vez, aunque en un anillo con flooding puro el paquete le llega por los dos caminos posibles, vía B y vía D: el segundo paquete, el que llega por el otro camino, se descarta silenciosamente en vez de imprimirse dos veces. Eso confirma que la deduplicación por id de mensaje funciona.

```
nodo A arriba en 127.0.0.1:6001, algoritmo=flooding, vecinos=['B', 'D', 'E']
comandos: send <destino> <mensaje> | table | quit
[]

nodo D arriba en 127.0.0.1:6004, algoritmo=flooding, vecinos=['A', 'C']
comandos: send <destino> <mensaje> | table | quit
[]

nodo C arriba en 127.0.0.1:6003, algoritmo=flooding, vecinos=['B', 'D']
comandos: send <destino> <mensaje> | table | quit
[C] mensaje de E: hola-desde-E-flooding
[]

nodo E arriba en 127.0.0.1:6005, algoritmo=flooding, vecinos=['A']
comandos: send <destino> <mensaje> | table | quit
[]

nodo B arriba en 127.0.0.1:6002, algoritmo=flooding, vecinos=['A', 'C']
comandos: send <destino> <mensaje> | table | quit
[]
```

LSR: convergencia y envío ruteado

Se levantaron los cinco nodos, se esperaron 7 segundos dos rondas de info, cada una cada 5 s para que los LSPs terminaran de propagarse, y se pidió la tabla en A, C y E. Las tablas coincidieron con el cálculo de Dijkstra standalone sobre la topología completa (ver la comparación de tablas de ruteo, más adelante), lo que confirma que la base de datos de enlaces armada dinámicamente por cada nodo terminó siendo igual a la topología real. El mensaje de E a C llegó por la tabla salto por salto, E ->A ->B ->C o E ->A ->D ->C, ambos de costo 2 no por broadcast como en flooding.

```
nodo A arriba en 127.0.0.1:6004, algoritmo=lsr, vecinos=['B', 'D', 'E']
comandos: send <destino> <mensaje> | table | quit
B → B
C → B
D → D
E → E
[]

nodo C arriba en 127.0.0.1:6003, algoritmo=lsr, vecinos=['B', 'D']
comandos: send <destino> <mensaje> | table | quit
A → B
B → B
D → D
E → B
[C] mensaje de E: hola-desde-E-via-LSR

nodo D arriba en 127.0.0.1:6004, algoritmo=lsr, vecinos=['A', 'C']
comandos: send <destino> <mensaje> | table | quit
[]

nodo E arriba en 127.0.0.1:6005, algoritmo=lsr, vecinos=['A']
comandos: send <destino> <mensaje> | table | quit
A → A
B → A
C → A
D → A
[]
```

DVR: convergencia y envío ruteado

Mismo procedimiento, esperando 12 segundos: DVR tarda más rondas en converger porque cada nodo sólo aprende sobre destinos que están un salto más lejos que lo que ya sabían sus vecinos en la ronda anterior. La información se propaga de a un salto por ronda, no de una vez como el LSP completo de LSR.

```
nodo A arriba en 127.0.0.1:6004, algoritmo=dvr, vecinos=['B', 'D', 'E']
comandos: send <destino> <mensaje> | table | quit
B → B
C → D
D → D
E → E
[]

nodo D arriba en 127.0.0.1:6004, algoritmo=dvr, vecinos=['A', 'C']
comandos: send <destino> <mensaje> | table | quit
[]

nodo E arriba en 127.0.0.1:6005, algoritmo=dvr, vecinos=['A']
comandos: send <destino> <mensaje> | table | quit
A → A
B → A
C → A
D → A

nodo C arriba en 127.0.0.1:6003, algoritmo=dvr, vecinos=['B', 'D']
comandos: send <destino> <mensaje> | table | quit
A → D
B → B
D → D
E → D
[C] mensaje de E: hola-desde-E-dvr
[]

nodo B arriba en 127.0.0.1:6002, algoritmo=dvr, vecinos=['A', 'C']
comandos: send <destino> <mensaje> | table | quit
[]
```

Un dato: A llega a C vía D en DVR, pero vía B en LSR. Ambos caminos cuestan lo mismo, dos saltos, así que no es un error cada algoritmo se quedó con el primer camino óptimo que le llegó primero, y el orden de llegada entre A-B y A-D no está garantizado. Esto se repitió al correr la prueba dos veces: en la primera corrida C llegaba a A (y a E) vía B, y en la segunda la de la captura vía D. Mismo costo, distinto ganador de la carrera entre rondas.

Comparación de tablas de ruteo

Dijkstra standalone corre en modo dijkstra, lee la topología completa directamente y calcula la tabla al arrancar, sin esperar convergencia. Sirve como referencia para comparar contra lo que LSR y DVR calculan de forma dinámica. La tabla siguiente compara el siguiente salto calculado por los tres algoritmos para los nodos A, C y E.

```

nodo A arriba en 127.0.0.1:6001, algoritmo=dijkstra, vecinos=['B', 'D', 'E']
comandos: send <destino> <mensaje> | table | quit
B → B
C → B
D → D
E → E
[]

nodo D arriba en 127.0.0.1:6004, algoritmo=dijkstra, vecinos=['A', 'C']
comandos: send <destino> <mensaje> | table | quit
A → A
B → A
C → C
E → A
[]

nodo C arriba en 127.0.0.1:6003, algoritmo=dijkstra, vecinos=['B', 'D']
comandos: send <destino> <mensaje> | table | quit
A → B
B → B
D → D
E → B
[]

nodo E arriba en 127.0.0.1:6005, algoritmo=dijkstra, vecinos=['A']
comandos: send <destino> <mensaje> | table | quit
A → A
B → A
C → A
D → A
[]

nodo B arriba en 127.0.0.1:6002, algoritmo=dijkstra, vecinos=['A', 'C']
comandos: send <destino> <mensaje> | table | quit
A → A
C → C
D → A
E → A
[]

```

Nodo origen	Destino	Dijkstra	LSR	DVR
A	B	B	B	B
A	C	B	B	D
A	D	D	D	D
A	E	E	E	E
C	A	B	B	D
C	B	B	B	B
C	D	D	D	D
C	E	B	B	D
E	A	A	A	A
E	B	A	A	A
E	C	A	A	A
E	D	A	A	A

Las tablas de A y E coinciden exactamente entre los tres algoritmos. En C, DVR difiere de Dijkstra y LSR en dos destinos A y E, ambos vía D en vez de vía B, consistente con lo observado en la prueba de DVR descrita arriba: mismo costo, camino distinto.

5. Discusión

Flooding garantiza que el mensaje llegue mientras exista algún camino sin necesidad de que la red converja primero, pero a costa de mandar copias del mismo mensaje por todos los caminos posibles. En la topología de prueba, el mensaje de E a C generó tráfico por ambos brazos del anillo aunque solo uno era necesario: es el algoritmo más simple de los tres, y el que menos escala en tráfico.

LSR converge más rápido y de forma más predecible: con dos rondas, 10 segundos, las tablas ya coincidían con el cálculo estático de Dijkstra. Tiene sentido: cada LSP viaja completo y llega

a todos los nodos por flooding, así que la base de datos de enlaces de cada nodo se completa en pocos saltos.

DVR necesitó más tiempo para estabilizarse se usaron 12 segundos en la prueba porque la información viaja de a un salto por ronda: un nodo recién se entera de un destino lejano cuando su vecino ya se enteró en la ronda anterior. En una red más grande esto se nota más.

Los tres algoritmos, una vez convergidos (o calculados, en el caso de Dijkstra), coinciden en el costo de las rutas, aunque no siempre en el siguiente salto elegido cuando hay más de un camino óptimo. Es un resultado correcto, no un error: ninguno de los tres desempata por ningún criterio adicional.

La limitación más importante que queda pendiente es la detección de nodos caídos: un nodo solo reacciona a lo que le llega, sin timeout de hello que retire a un vecino que dejó de responder. Para la parte de pruebas en vivo del laboratorio, con nodos cayéndose y volviendo, esto habría que agregarlo.